

**Portada****Nombre del proyecto:**

ReservaFacil – Plataforma Web de Gestión de Reservas para Barbería

Nombre del equipo:**Integrantes:**

- Claudio González

Sección:

TALLER APLICADO DE PROGRAMACION_001D

Docente:

CARLOS ENRIQUE QUINTANILLA GALLEGUILLOS

Fecha de entrega:

27-05-2026

Índice

1. Introducción
2. Resumen del proyecto definido en Fase 1
3. Problema u oportunidad abordada
4. Objetivo general y objetivos específicos
5. Alcance actualizado del proyecto
6. Metodología de trabajo aplicada
7. Gestión Scrum del proyecto
8. Historias de usuario y Product Backlog
9. Sprint Backlog y planificación de desarrollo
10. Diseño de la solución
11. Arquitectura técnica propuesta
12. Modelo de datos y base de datos
13. Ambiente de pruebas
14. Configuración del servidor o entorno Cloud
15. Procedimiento de respaldo y restauración de base de datos
16. Avance funcional del producto
17. Evidencia de desarrollo en GitHub
18. Plan de pruebas y pruebas ejecutadas
19. Estado de avance del MVP
20. Riesgos, problemas y acciones correctivas
21. Conclusiones
22. Lecciones aprendidas
23. Anexos

El presente informe corresponde al Estado de Avance Fase 2 del proyecto ReservaFacil, desarrollado en el contexto de la asignatura TPY1101 – Taller Aplicado de Programación.

El propósito es evidenciar el avance alcanzado durante esta etapa del desarrollo, considerando la planificación y definición inicial realizadas en la Fase 1, junto con la implementación progresiva del producto utilizando una metodología ágil basada en Scrum.

Durante esta fase se ejecutaron actividades relacionadas con el desarrollo técnico, metodológico y funcional del sistema, incorporando procesos de análisis, diseño, programación, pruebas y despliegue del producto en ambientes cloud. Asimismo, se documentó la arquitectura de la solución, la configuración del entorno de ejecución, la gestión del trabajo mediante herramientas colaborativas y el control de versiones utilizando GitHub.

Como resultado, el informe presenta evidencia concreta del avance del producto mínimo viable (MVP), incluyendo funcionalidades implementadas, integración entre componentes, validaciones realizadas y acciones adoptadas para resolver incidencias detectadas durante el proceso de construcción del sistema.

2. Resumen del proyecto definido en Fase 1

Deben resumir lo definido en la primera fase del proyecto.

Elemento	Descripción
Nombre del proyecto	ReservaFacil – Plataforma Web de Gestión de Reservas para Barbería
Cliente, usuario o beneficiario	Clientes de la barbería y administrador del negocio, quienes requieren una solución digital para gestionar reservas y mejorar el proceso de atención
Problema u oportunidad detectada	El proceso de reservas se realizaba de forma manual mediante mensajes o coordinación directa, generando errores en horarios, baja organización, pérdida de información y poca trazabilidad del servicio
Solución propuesta	Desarrollo de una plataforma web que permita registrar usuarios, iniciar sesión, gestionar reservas, realizar pagos electrónicos y centralizar la administración del servicio
Tipo de solución	Web
Tecnologías inicialmente definidas	HTML5, CSS3, JavaScript, Node.js, Stripe, GitHub, Render y django
Resultado esperado	Implementar una plataforma funcional capaz de gestionar reservas de manera digital, mejorar la experiencia del usuario, optimizar la organización del negocio y disponer de un sistema desplegado en ambiente cloud

3. Problema u oportunidad abordada

El proyecto ReservaFacil surge como respuesta a una problemática identificada en el proceso de gestión de reservas utilizado por barberías que operan mediante mecanismos tradicionales de atención.

Actualmente, gran parte del proceso de reserva se realiza de forma manual mediante mensajes, llamadas telefónicas o coordinación directa entre cliente y administrador, provocando dificultades para mantener un control organizado de horarios y disponibilidad.

Esta situación afecta principalmente a dos grupos:

Clientes, quienes presentan dificultades para consultar disponibilidad, coordinar horarios y confirmar servicios de manera rápida.

Administrador o encargado del negocio, quien debe gestionar manualmente reservas, disponibilidad y seguimiento de clientes.

Como consecuencia de este proceso tradicional se generan diversos inconvenientes, entre ellos:

- Duplicidad o pérdida de reservas.
- Errores en horarios de atención.
- Baja trazabilidad del historial de clientes.
- Mayor tiempo destinado a coordinación manual.
- Limitaciones para automatizar pagos y confirmaciones.

Frente a esta necesidad, se determinó que una solución basada en software permitiría digitalizar y centralizar el proceso de atención mediante una plataforma web accesible, automatizando el registro de usuarios, autenticación, reservas y pagos electrónicos, mejorando tanto la experiencia del cliente como la gestión operativa del servicio.

La validación del problema se realizó mediante análisis del proceso actual de reservas, observación del flujo de atención y levantamiento de requerimientos asociados a la experiencia esperada por usuarios de servicios de este tipo. A partir de este análisis se definieron las funcionalidades principales que dieron origen al producto desarrollado durante el proyecto.

4. Objetivo general y objetivos específicos

Objetivo general

Desarrollar una plataforma web denominada ReservaFacil, orientada a la gestión digital de reservas para servicios de barbería, permitiendo a los usuarios registrar sus datos, autenticarse, seleccionar servicios, gestionar reservas y realizar pagos electrónicos, con el propósito de mejorar la experiencia del cliente y optimizar la administración operativa del negocio.

Objetivos específicos

- Implementar un sistema de registro e inicio de sesión que permita administrar el acceso de los usuarios a la plataforma.
- Desarrollar un módulo de reservas que permita seleccionar servicios, fechas y horarios disponibles de manera organizada y centralizada.
- Integrar un sistema de pagos electrónicos mediante Stripe para automatizar el proceso de confirmación de reservas.
- Implementar el despliegue del sistema en ambiente cloud utilizando herramientas de publicación y control de versiones que permitan disponibilidad el producto para pruebas y validación funcional.

5. Alcance actualizado del proyecto

Durante el desarrollo de la Fase 2 se mantuvo el objetivo principal definido inicialmente; sin embargo, se realizaron ajustes en el alcance funcional del producto para priorizar el desarrollo del MVP (Producto Mínimo Viable) y asegurar una versión funcional y demostrable dentro del tiempo disponible

Elemento	Definición
Alcance funcional incluido	Desarrollo de una plataforma web para registro de usuarios, autenticación, gestión de reservas, selección de servicios, integración de pagos y despliegue en ambiente cloud
Funcionalidades que serán desarrolladas en el MVP	Registro de usuarios, inicio de sesión, reserva de servicios, selección de fecha y horario, visualización de reservas, notificaciones básicas, integración de pagos y despliegue funcional
Funcionalidades descartadas o postergadas	Panel administrativo avanzado, reportes estadísticos, recuperación automática de contraseña, gestión avanzada de clientes y mejoras visuales secundarias
Justificación de cambios respecto a Fase 1	Durante el desarrollo se priorizaron funcionalidades críticas para asegurar un producto funcional e integrado. Algunas funcionalidades complementarias fueron reprogramadas para Fase 3 con el objetivo de concentrar esfuerzos en estabilidad, integración y despliegue operativo
Restricciones técnicas o de tiempo	Tiempo limitado de desarrollo académico, necesidad de integración entre frontend y backend, configuración de servicios cloud, pruebas de integración y adaptación de servicios externos como Stripe

6. Metodología de trabajo aplicada

Para el desarrollo del proyecto ReservaFacil se utilizó el marco de trabajo ágil Scrum, con el objetivo de organizar el trabajo del equipo, distribuir responsabilidades, realizar seguimiento continuo del avance y entregar incrementos funcionales del producto durante cada etapa del desarrollo.

La aplicación de Scrum permitió planificar tareas mediante backlog, asignar responsables, controlar avances y validar funcionalidades terminadas mediante revisiones periódicas del producto.

Durante la Fase 2 el equipo trabajó mediante iteraciones cortas orientadas al cumplimiento del MVP, manteniendo registro de actividades, evidencias técnicas y control de versiones.

Elemento Scrum	Evidencia esperada
Product Owner	Claudio Gonzalez. Responsable de definir prioridades del producto, validar funcionalidades y mantener alineado el desarrollo con los objetivos definidos
Scrum Máster	Claudio González. Responsable del seguimiento del proyecto, coordinación del equipo, control del cumplimiento de tareas y apoyo en resolución de bloqueos técnicos
Dev Team	Claudio González- Responsables del análisis, desarrollo frontend, backend, integración, pruebas y documentación
Product Backlog	https://github.com/Claugonzalez21v/CAPSTONE_001D/blob/ef708cdba71520f3a7068974e6348849b91f010e/Documentacion/3.-%20plantilla_product_backlog_puntos_historia.xlsx
Sprint Backlog	https://github.com/Claugonzalez21v/CAPSTONE_001D/blob/ef708cdba71520f3a7068974e6348849b91f010e/Documentacion/4.1-%20plantilla_sprint_backlog_actividades_horas%20-%20Sprint%201.xlsx
Sprint Review	https://github.com/Claugonzalez21v/CAPSTONE_001D/tree/2f909aedac59ca1c31378040d465c6f2bbb19cd/Documentacion/V%20validacion%20de%20sprint

Elemento Scrum	Evidencia esperada
Daily o seguimiento	https://github.com/Claugonzalez21v/CAPSTONE_001D/blob/ef708cdba71520f3a7068974e6348849b91f010e/Documentacion/9.-%20Documento%20retrospectiva%20del%20proyecto.docx
Herramienta utilizada	Trello, GitHub

7. Gestión Scrum del proyecto

Deben presentar cómo organizaron, planificaron y ejecutaron el trabajo.

7.1 Herramienta de gestión utilizada

Herramienta:

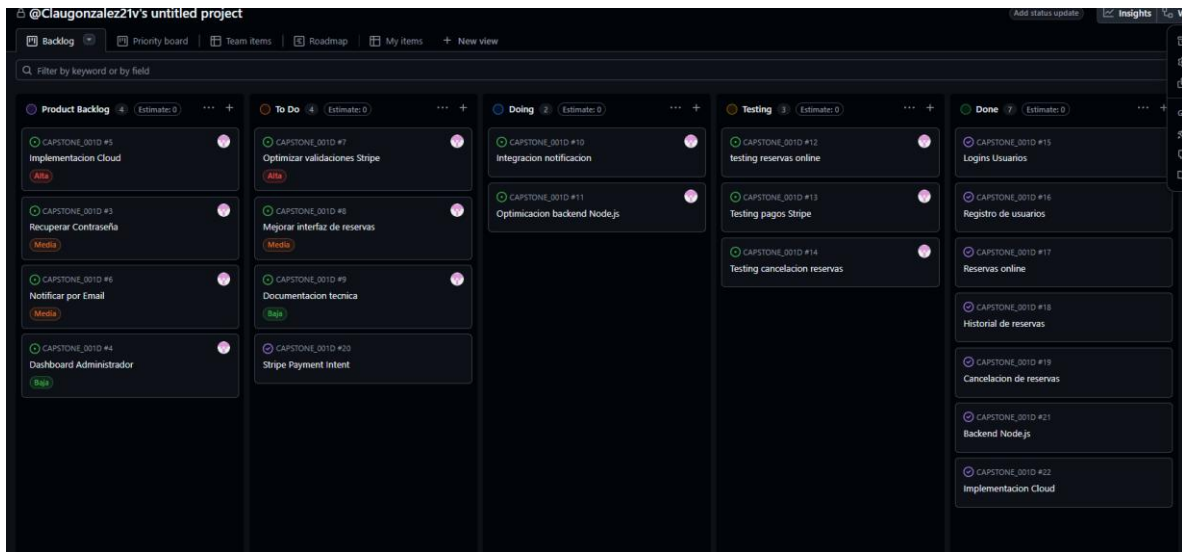
GitHub Projects + GitHub

Enlace al tablero:

<https://github.com/users/Claugonzalez21v/projects/3/views/1>

7.2 Evidencia del tablero

Imagen / captura:



7.3 Organización del trabajo

El proyecto fue desarrollado utilizando GitHub Projects como herramienta de gestión Scrum. Las tareas fueron organizadas en columnas según su estado de avance: Product Backlog, Sprint Backlog, En Desarrollo, En Revisión, En Pruebas y Terminado.

Claudio González asumió los roles de Product Owner, Scrum Máster y Desarrollador, realizando la planificación, implementación, pruebas y documentación del sistema.

El seguimiento se efectuó mediante revisiones periódicas del tablero, actualización de tareas y control de versiones utilizando GitHub.

8. Historias de usuario y Product Backlog

Deben incluir las historias de usuario principales del proyecto.

ID	Historia de usuario	Prioridad	Criterio de aceptación	Estado
HU-01	Como usuario quiero registrarme para acceder al sistema	Alta	El sistema permite crear una cuenta y almacenar correctamente la información ingresada	Terminada
HU-02	Como usuario quiero iniciar sesión para acceder a mis funcionalidades y reservas	Alta	El sistema valida credenciales y permite acceso al usuario registrado	Terminada
HU-03	Como usuario quiero reservar un servicio seleccionando fecha y horario para gestionar mi atención	Alta	El sistema registra correctamente la reserva y muestra confirmación	Terminada
HU-04	Como usuario quiero realizar el pago del servicio para confirmar mi reserva	Media	El sistema procesa el pago y confirma el estado de la reserva	Terminada

Cada historia debe estar vinculada a una necesidad del usuario y a una funcionalidad del producto.

9. Sprint Backlog y planificación de desarrollo

Deben mostrar cómo organizaron las historias en Sprint.

Sprint	Fecha inicio	Fecha término	Historias incluidas	Responsable(s)	Estado
Sprint 1	01-04-2026	14-04-2026	HU-01, HU-02	Claudio Gonzalez	Cerrado
Sprint 2	15-04-2026	30-04-2026	HU-03, HU-04	Claudio Gonzalez	Cerrado
Sprint 3	01-05-2026	19-05-2026	HU-05, HU-06,	Claudio González	Cerrado

9.1 Trazabilidad historia → tarea → commit

Esta tabla es clave para la defensa.

Hist oria de usua rio	Tarea en tablero	Respon sable	Fecha término	Commit GitHub	Evidencia
HU-01	Implemen tación módulo registro usuarios	Claudio González	08-06-2026	2d1b86bb3646d ef67e8375a3f45 f31db3cbfc891	https://github.com/ClaudioGonzalez21v/CAPSTONE_001D/blob/2f998aedac59ca1c313780405465c8f2bbba19cd/Documentacion/2.-%20Historias%20Usuario.xlsx
HU-02	Desarrollo autentica ción e inicio sesión	Claudio González	08-06-2026	2d1b86bb3646d ef67e8375a3f45 f31db3cbfc891	https://github.com/ClaudioGonzalez21v/CAPSTONE_001D/blob/2f998aedac59ca1c313780405465c8f2bbba19cd/Documentacion/2.-%20Historias%20Usuario.xlsx

Historia de usuario	Tarea en tablero	Responsable	Fecha término	Commit GitHub	Evidencia
HU-03	Desarrollo módulo de reservas	Claudio González	08-06-2026	2d1b86bb3646def67e8375a3f45f31db3cbfc891	https://github.com/Claugonzalez21v/CAPSTONE_001D/blob/2f909baedac58ca1c31378040d465e8f2bbba19cd/Documentacion/2.-%20Historias%20Usuario.xlsx
HU-04	Implementación agenda e historial de reservas	Claudio González	08-06-2026	2d1b86bb3646def67e8375a3f45f31db3cbfc891	https://github.com/Claugonzalez21v/CAPSTONE_001D/blob/2f909baedac58ca1c31378040d465e8f2bbba19cd/Documentacion/2.-%20Historias%20Usuario.xlsx
HU-05	Integración de pagos Stripe Sandbox	Claudio González	08-06-2026	2d1b86bb3646def67e8375a3f45f31db3cbfc891	https://github.com/Claugonzalez21v/CAPSTONE_001D/blob/2f909baedac58ca1c31378040d465e8f2bbba19cd/Documentacion/2.-%20Historias%20Usuario.xlsx
HU-06	Despliegue cloud en Render	Claudio González	08-06-2026	2d1b86bb3646def67e8375a3f45f31db3cbfc891	https://github.com/Claugonzalez21v/CAPSTONE_001D/blob/2f909baedac58ca1c31378040d465e8f2bbba19cd/Documentacion/2.-%20Historias%20Usuario.xlsx

10. Diseño de la solución

Deben incorporar los diagramas y documentos que permitan comprender cómo será construida la solución.

Pueden incluir:

- Diagrama de casos de uso.
- Diagrama de arquitectura.
- Diagrama entidad-relación.
- Diagrama de flujo del proceso.
- Mockups o prototipos.
- Modelo de navegación.
- Historias de usuario.
- Reglas de negocio.

Los diagramas los quedaron documentados en una carpeta aparte para su visualización.

Enlace de ingreso:

https://github.com/Claugonzalez21v/CAPSTONE_001D/tree/2f909aedac59ca1c31378040d465c6f2bbbba19cd/Documentacion/Diagramas

10.1 Diagramas incorporados

Diagrama / documento	Propósito	Estado
Diagrama de casos de uso	Representar interacción usuario–sistema	Completo
Diagrama de arquitectura	Mostrar componentes técnicos	Completo
Modelo de base de datos	Representar entidades y relaciones	Completo
Mockups	Representar interfaz del sistema	Completo

11. Arquitectura técnica propuesta

La arquitectura técnica del proyecto ReservaFacil fue diseñada utilizando un enfoque cliente–servidor, separando la interfaz de usuario (Frontend) de la lógica de negocio - (Backend), permitiendo mejorar la mantenibilidad, facilitar el despliegue y soportar futuras ampliaciones del sistema.

La solución fue desplegada en ambiente cloud para permitir acceso remoto y validación funcional del producto.

Componente	Tecnología utilizada	Descripción
Frontend	HTML5 + CSS3 + JavaScript	Responsable de la interfaz gráfica del sistema, formularios, navegación e interacción del usuario.
Backend	Django 6 (Python 3.11)	Responsable del procesamiento de solicitudes, autenticación de usuarios, gestión de reservas e integración con servicios externos.
Base de datos	SQLite (Django)	Sistema de almacenamiento persistente utilizado para guardar usuarios, reservas y datos del sistema.
Autenticación	Django Sessão	Sistema de autenticación mediante sesiones proporcionado por Django para controlar el acceso de usuarios.
Pasarela de pagos	Stripe Sandbox	Plataforma utilizada para procesar pagos electrónicos en entorno de pruebas.
Hosting / Cloud	Render	Servicios utilizados para publicar el sistema y permitir acceso mediante internet.
Control de versiones	GitHub	Plataforma utilizada para almacenar código fuente, registrar commits y mantener trazabilidad del desarrollo.
Entorno de desarrollo	Visual Studio Code	Editor de código utilizado para el desarrollo y mantenimiento del proyecto.

11.1 Justificación técnica

Las tecnologías seleccionadas fueron elegidas considerando facilidad de integración, velocidad de desarrollo y compatibilidad con despliegue cloud.

El uso de Django 6 sobre Python 3.11 permitió implementar la lógica de negocio del sistema, gestionar autenticación de usuarios, reservas, calendario e integración con Stripe. Su arquitectura facilita el mantenimiento del código, la seguridad y el despliegue en la nube mediante Render.

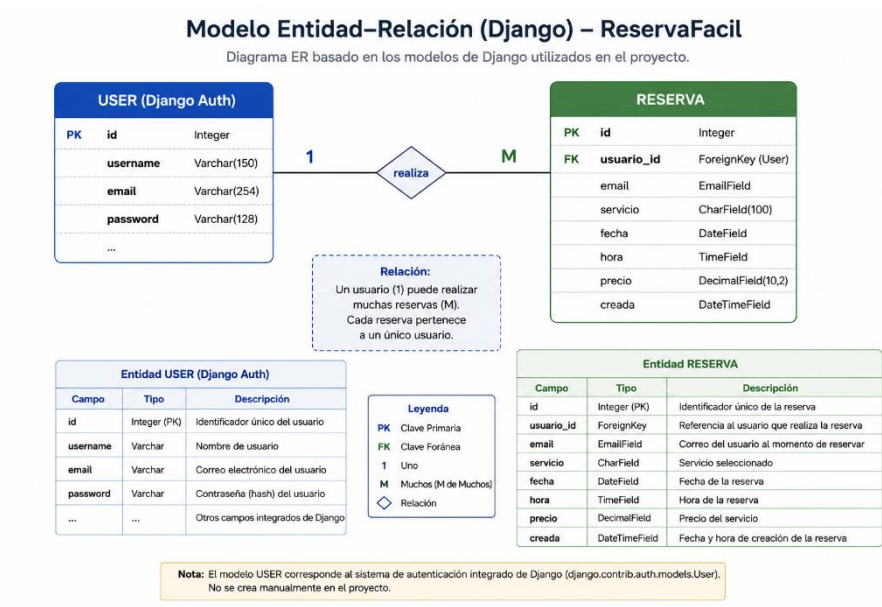
Para el despliegue se utilizaron Render para la publicación del frontend y Render para el alojamiento del backend, permitiendo ejecutar pruebas funcionales en un entorno cercano a producción.

GitHub fue utilizado como mecanismo principal de control de versiones, permitiendo mantener trazabilidad entre historias de usuario, tareas y código desarrollado.

12. Modelo de datos y base de datos

Deben presentar el diseño de la base de datos.

12.1 Modelo entidad-relación



12.2 Diccionario de datos

Tabla	Campo	Tipo de dato	Descripción	Restricciones
usuarios	id_usuario	int	Identificador único del usuario	PK
usuarios	nombre	varchar	Nombre del usuario registrado	Obligatorio
usuarios	correo	varchar	Correo electrónico del usuario	Obligatorio / único
usuarios	contraseña	varchar	Contraseña del usuario	Obligatorio
usuarios	id_usuario	int	Identificador único del usuario	PK

12.3 Script o evidencia de creación

12.3 Script o evidencia de creación

Durante la Fase 2 se implementó la estructura de datos utilizando el ORM de Django, permitiendo la creación automática de tablas mediante modelos definidos en Python. La persistencia de la información se realiza utilizando SQLite integrado por defecto en Django.

Modelo Reserva (models.py)

```

from django.db import models
from django.contrib.auth.models import User

class Reserva(models.Model):
    usuario = models.ForeignKey(
        User,
        on_delete=models.CASCADE
    )
    email = models.EmailField(
        blank=True,
        null=True
    )
    servicio = models.CharField(
        max_length=100
    )
    fecha = models.DateField()
    hora = models.TimeField()
    precio = models.DecimalField(
        max_digits=10,
        decimal_places=2
    )
    creada = models.DateTimeField(
        auto_now_add=True
    )

    def __str__(self):
        return f'Reserva de {self.usuario.username} - {self.servicio}'

```

Notas

- El modelo Reserva utiliza como referencia al modelo User de Django (django.contrib.auth.models.User).
- Las tablas son creadas automáticamente mediante migraciones.
- No se requirió creación manual de scripts SQL.

Migraciones ejecutadas

```

$ python manage.py makemigrations
Migrations for 'reservas':
  reservas/migrations/0001_initial.py
  - Create model Reserva

$ python manage.py migrate

Operations to perform:
  Apply all migrations: admin, auth, contenttypes, sessions, reservas
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying auth.0009_alter_user_last_name_max_length... OK
  Applying auth.0010_alter_group_name_max_length... OK
  Applying auth.0011_update_proxy_permissions... OK
  Applying sessions.0001_initial... OK
  Applying reservas.0001_initial... OK

```

Estructura creada en Django

La tabla se crea automáticamente según el modelo Reserva definido.

Modelo: Reserva

Campo	Tipo	Descripción
id	AutoField (PK)	Identificador único
usuario_id	ForeignKey (User)	Referencia al usuario
email	EmailField	Correo del usuario
servicio	CharField (100)	Servicio seleccionado
fecha	DateField	Fecha de la reserva
hora	TimeField	Hora de la reserva
precio	DecimalField (10,2)	Precio del servicio
creada	DateTimeField	Fecha y hora de creación

Archivo generado

Durante las migraciones se genera la base de datos SQLite del proyecto de Django.

db.sqlite3

Evidencia en Django Admin

Django administration

3 items to be shown.

Authentication and Authorization

Groups

Users

Reservas

Reservas

Select reserva to change

Action

Go

3 of 3 selected

	ID	USUARIO	EMAIL	SERVICIO	FECHA	HORA	PRECIO	CREADA
☐	3	eladio	eladio@empresa.com	Corte de Cabello	May 27, 2026	15:00:00	10000.00	May 27, 2026, 19:15 a.m.
☐	2	eladio	eladio@empresa.com	Buena	May 27, 2026	18:00:00	8000.00	May 27, 2026, 19:10 a.m.
☐	1	eladio	eladio@empresa.com	Corte de Cabello	May 26, 2026	14:00:00	10000.00	May 26, 2026, 09:30 a.m.

3 reservas

13. Ambiente de pruebas

Deben explicar el ambiente donde se está probando el sistema.

Elemento	Descripción
Tipo de ambiente	Local y Cloud
Sistema operativo	Windows 11
Lenguaje utilizado	JavaScript, Python, HTML, CSS
Framework / entorno	Django 6
Base de datos	SQLite (Django)
URL de prueba frontend	https://capstone-001d.onrender.com
Hosting Cloud	Render
Control de versiones	GitHub
Pasarela de pagos	Stripe Sandbox

El instrumento oficial solicita evidenciar una configuración que permita replicar o acercarse al ambiente de producción, incluyendo pruebas operativas, validación y verificación.

14. Configuración del servidor o entorno Cloud

Para la ejecución y validación del proyecto ReservaFacil se configuró un ambiente cloud compuesto por servicios separados para frontend y backend.

La arquitectura implementada permitió publicar el sistema para acceso remoto, realizar pruebas funcionales y mantener integración continua mediante GitHub.

14.1 Instrucciones de instalación

Ejemplo:

1. Clonar repositorio desde GitHub.
2. Instalar dependencias.
3. Configurar variables de entorno.
4. Crear base de datos.
5. Ejecutar script de carga inicial.
6. Levantar servidor backend.
7. Levantar frontend.
8. Ejecutar pruebas básicas.

14.2 Comandos principales

- `git clone https://github.com/Claugonzalez21v/CAPSTONE_001D.git`
- `cd CAPSTONE_001D`
- `pip install -r requirements.txt`
- `python manage.py migrate`
- `python manage.py collectstatic --noinput`
- `python manage.py runserver`

14.3 Variables de entorno

Variable	Descripción	Ejemplo
STRIPE_SECRET_KEY	Clave secreta de Stripe	sk_test_XXXXXXXXXX
EMAIL_USER	Correo utilizado para envío de confirmaciones	barberiashop323@gmail.com
EMAIL_PASS	Contraseña de aplicación de Gmail	*****
PORT	Puerto de ejecución	PORT=3000

No deben incluir claves reales, contraseñas ni tokens sensibles en el informe.

15. Procedimiento de respaldo y restauración de base de datos

Durante la Fase 2, el sistema ReservaFacil utiliza una base de datos SQLite gestionada por Django. Por esta razón, el procedimiento de respaldo y restauración se realiza mediante la copia y recuperación del archivo de base de datos db.sqlite3, el cual contiene toda la información de usuarios, reservas y datos asociados al sistema.

15.1 Procedimiento de respaldo

1. Detener servidor Django.
2. Ubicar archivo db.sqlite3.
3. Crear copia de seguridad.
4. `copy db.sqlite3 respaldo_db.sqlite3`

```
copy db.sqlite3 respaldo_db.sqlite3
restauración:
copy respaldo_db.sqlite3 db.sqlite3
```

15.2 Procedimiento de restauración

1. Detener la ejecución del servidor Django.
2. Ubicar el archivo de respaldo generado previamente (respaldo_db.sqlite3).
3. Acceder a la carpeta principal del proyecto donde se encuentra la base de datos actual.
4. Reemplazar el archivo de base de datos existente por el archivo de respaldo utilizando el siguiente comando:

`copy respaldo_db.sqlite3 db.sqlite3`
5. Verificar que el archivo haya sido reemplazado correctamente.
6. Ejecutar nuevamente las migraciones si corresponde:

`Python manage.py migrate`
7. Iniciar el servidor Django:

`Python manage.py runserver`
8. Ingresar al sistema ReservaFacil.
9. Validar que los usuarios, reservas y demás registros se encuentren disponibles correctamente.
10. Realizar una prueba funcional de inicio de sesión y consulta de reservas para confirmar la integridad de la información **restaurada**.

15.3 Evidencia

Para validar el procedimiento de respaldo y restauración se realizaron pruebas utilizando la base de datos SQLite implementada en Django. La evidencia considera la creación de una copia de seguridad del archivo db.sqlite3, su restauración y la posterior validación funcional mediante inicio de sesión y consulta de reservas.

Actividad	Evidencia	Responsable
Respaldo BD	[Captura/link]	[Nombre]
Restauración BD	[Captura/link]	[Nombre]

16. Avance funcional del producto

Durante la Fase 2 se desarrolló el Producto Mínimo Viable (MVP) del sistema ReservaFacil, implementando las funcionalidades críticas para validar el flujo principal del usuario.

El avance funcional fue medido considerando desarrollo, integración, pruebas y disponibilidad del producto.

Funcionalidad	Historia asociada	Estado	Evidencia
Registro de usuario	HU-01	Completa	https://github.com/Claugonzalez21v/CAPSTONE_001D/tree/2f909aedac59ca1c31378040d465c6f2bbba19cd/Evidencias
Inicio de sesión	HU-02	Completa	https://github.com/Claugonzalez21v/CAPSTONE_001D/tree/2f909aedac59ca1c31378040d465c6f2bbba19cd/Evidencias
Visualización servicios	HU-03	Completa	https://github.com/Claugonzalez21v/CAPSTONE_001D/tree/2f909aedac59ca1c31378040d465c6f2bbba19cd/Evidencias
Gestión de reservas	HU-04	Completa	https://github.com/Claugonzalez21v/CAPSTONE_001D/tree/2f909aedac59ca1c31378040d465c6f2bbba19cd/Evidencias
Pago electrónico Stripe	HU-05	Parcial	
Historial reservas	HU-06	Completa	https://github.com/Claugonzalez21v/CAPSTONE_001D/tree/2f909aedac59ca1c31378040d465c6f2bbba19cd/Evidencias

Funcionalidad	Historia asociada	Estado	Evidencia
Despliegue Cloud	HU-07	Completa	https://github.com/Claugonzalez21v/CAPSTONE_001D/tree/2f909aedac59ca1c31378040d465c6f2bbba19cd/Evidencias

16.1 Viaje del cliente / usuario

El recorrido del usuario fue diseñado para representar el flujo principal del sistema ReservaFacil, permitiendo validar el funcionamiento del MVP y demostrar la experiencia completa durante la Sprint Review.

El objetivo del recorrido es mostrar cómo un usuario interactúa con el sistema desde el ingreso hasta la confirmación de una reserva.

Flujo del usuario

El usuario ingresa a la plataforma web mediante navegador.

Se registra utilizando sus datos personales o inicia sesión si ya posee cuenta.

Accede al panel principal y visualiza los servicios disponibles.

Selecciona el servicio que desea reservar.

Escoge fecha y horario disponible.

Confirma el registro de la reserva.

Realiza el proceso de pago electrónico.

Recibe confirmación del proceso realizado.

Consulta el historial o estado de sus reservas.

Al finalizar el proceso el usuario debe haber completado exitosamente la reserva y disponer de confirmación visible dentro del sistema.

17. Evidencia de desarrollo en GitHub

Deben incluir evidencia del repositorio.

Elemento	Link / evidencia
Repositorio principal	https://github.com/Claugonzalez21v/CAPSTONE_001D.git
Rama principal	
Ramas de trabajo	CAPSTONE_001D/Producto /Producto/
Commits relevantes	https://github.com/Claugonzalez21v/CAPSTONE_001D/commit/e33f2ad47fb04476406185fd0524d9c238de8731
Pull requests, si aplica	[Links]
README actualizado	Sí
Instrucciones de instalación	Sí

17.1 Commits por integrante

Integrante	Commits realizados	Funcionalidades trabajadas	Evidencia
Claudio González	25	Registro de usuarios, inicio de sesión, reservas, Stripe, despliegue cloud	Commit principal GitHub
Claudio González	8	Mejoras de interfaz y validaciones	Historial GitHub
Claudio González	12	Integración frontend-backend y pruebas	Historial GitHub

18. Plan de pruebas y pruebas ejecutadas

Con el objetivo de validar el funcionamiento del sistema ReservaFacil, se definió un plan de pruebas orientado a verificar autenticación, reservas, integración y despliegue del producto.

Las pruebas fueron ejecutadas sobre ambiente local y ambiente cloud.

ID prueba	Funcionalidad	Tipo de prueba	Resultado esperado	Resultado obtenido	Estado
PR-01	Inicio de sesión	Funcional	Usuario accede con credenciales válidas	Acceso correcto al sistema	Aprobada
PR-02	Registro	Validación	Sistema valida campos obligatorios	Validaciones ejecutadas correctamente	Aprobada
PR-03	Reserva	Funcional	Usuario registra reserva correctamente	Reserva almacenada exitosamente	Aprobada
PR-04	eliminación de reserva	Integración	El sistema debe eliminar la reserva	El sistema elimina la reserva	Aprobada
PR-05	Pago Stripe	Integración	Sistema procesa pago	Pago parcialmente integrado	Aprobada
PR-06	Historial de reservas	Integración	Muestra la reserva creada	Reserva creada	Aprobada
PR-07	Deploy Render	integración	Subir el sistema a la Cloud	Sistema subido a Cloud	Aprobada

Tipos de prueba sugeridos:

- Prueba funcional.
- Prueba de validación.
- Prueba de verificación.
- Prueba de seguridad básica.
- Prueba de base de datos.
- Prueba de usabilidad básica.

19. Estado de avance del MVP

Deben indicar el porcentaje de avance real del producto mínimo viable.

Área	Avance estimado	Evidencia
Documentación	95%	Informe y anexos
Frontend	95%	Sistema funcionando
Backend	95%	API operativo
Base de datos	90%	LocalStorage implementado
Integración frontend-backend	90%	Flujo operativo
Pruebas	90%	Registro pruebas
Despliegue / ambiente	100%	Rende

Estado general del MVP

Actualmente el producto dispone de funcionalidades principales implementadas y operativas: autenticación, reservas, navegación, despliegue cloud y pruebas funcionales.

Se encuentran parcialmente desarrolladas las integraciones de pago y mejoras complementarias que serán fortalecidas durante Fase 3.

20. Riesgos, problemas y acciones correctivas

Riesgo o problema	Impacto	Acción correctiva	Responsable	Estado
Problemas de configuración en Render	Alto	Reconfiguración del servicio y variables de entorno	Claudio González	Cerrado
Integración parcial de Stripe	Medio	Ajustes de API y pruebas de pago	Claudio González	Controlado
Cambios de alcance durante el desarrollo	Medio	Priorización del MVP	Claudio González	Controlado
Tiempo limitado para pruebas finales	Alto	Planificación de pruebas funcionales prioritarias	Claudio González	Controlado

Ejemplos:

- Retraso en desarrollo backend.
- Falta de integración con base de datos.
- Baja participación de un integrante.
- Problemas con despliegue Cloud.
- Cambios de alcance.
- Falta de validación con cliente real.

21. Conclusiones

Al finalizar la Fase 2 se logró desarrollar una versión funcional del proyecto ReservaFacil-BarberShop, permitiendo implementar y validar las funcionalidades principales definidas para el MVP. El sistema permite el registro e inicio de sesión de usuarios, gestión de reservas, visualización de agenda, almacenamiento persistente mediante SQLite e integración de pagos utilizando Stripe Sandbox.

Entre los avances más relevantes destacan el desarrollo de la interfaz web utilizando HTML5, CSS3 y JavaScript, la implementación del backend con Django 6 sobre Python 3.11, la integración de la base de datos SQLite y el despliegue exitoso de la aplicación en la plataforma cloud Render.

La aplicación de la metodología Scrum permitió organizar el trabajo en iteraciones, facilitando el seguimiento de tareas, la gestión del Product Backlog y la validación continua de funcionalidades durante el desarrollo del proyecto. Asimismo, el uso de GitHub permitió mantener el control de versiones y la trazabilidad de los cambios implementados.

Los resultados obtenidos permiten concluir que el sistema cumple satisfactoriamente con los requerimientos funcionales establecidos para esta fase, demostrando la viabilidad técnica de la solución propuesta.

Para una futura evolución del proyecto se contempla optimizar la experiencia de usuario, ampliar la cobertura de pruebas, incorporar funcionalidades administrativas avanzadas y fortalecer la integración de servicios complementarios que permitan enriquecer la plataforma.

22. Lecciones aprendidas

Cada integrante debe incluir una breve reflexión individual.

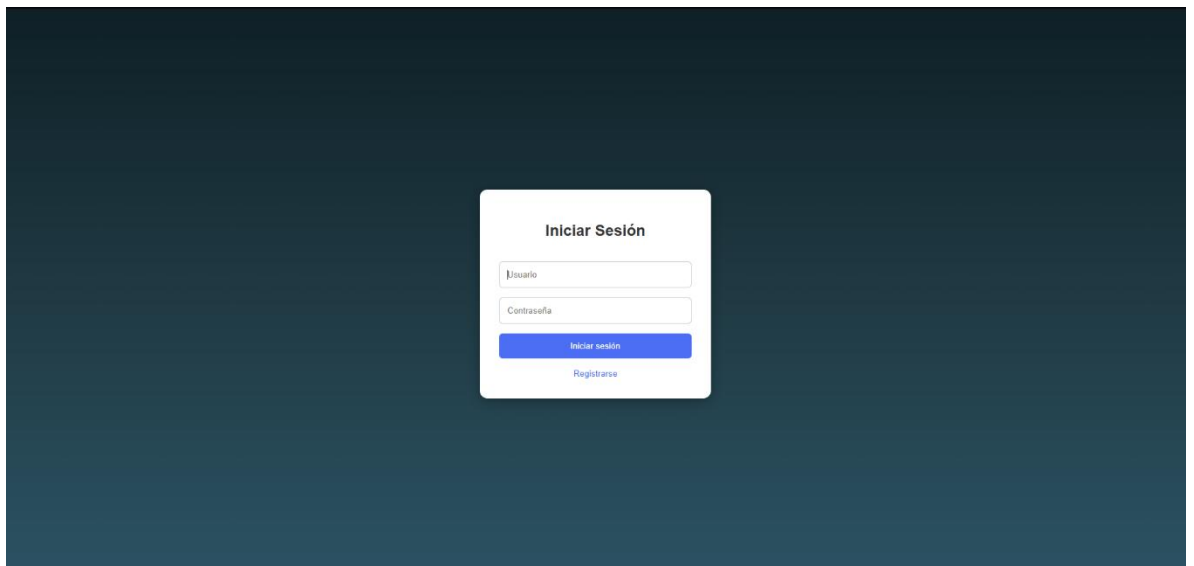
Integrante	Lección aprendida	Aporte realizado
Claudio González	La organización iterativa permitió controlar mejor el desarrollo y resolver incidencias progresivamente	Desarrollo frontend, backend, integración cloud, documentación y pruebas

23. Anexos

Incluir evidencias adicionales:

Evidencia funcional

- Captura Login.



A screenshot of a login form titled "Iniciar Sesión" centered on a dark blue gradient background. The form is a white card with a title, two input fields for "Usuario" and "Contraseña", a blue "Iniciar sesión" button, and a link "Regístrate" below the button.

Iniciar Sesión

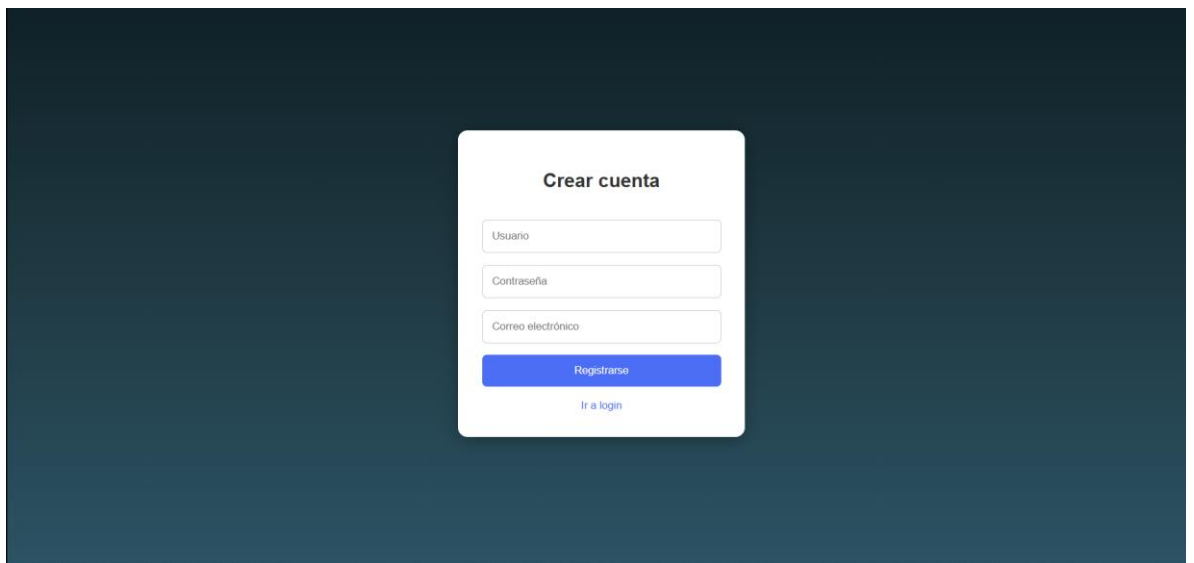
Usuario

Contraseña

Iniciar sesión

[Regístrate](#)

- Captura Registro.



A screenshot of a registration form titled "Crear cuenta" centered on a dark blue gradient background. The form is a white card with a title, three input fields for "Usuario", "Contraseña", and "Correo electrónico", a blue "Registrarse" button, and a link "Ir a login" below the button.

Crear cuenta

Usuario

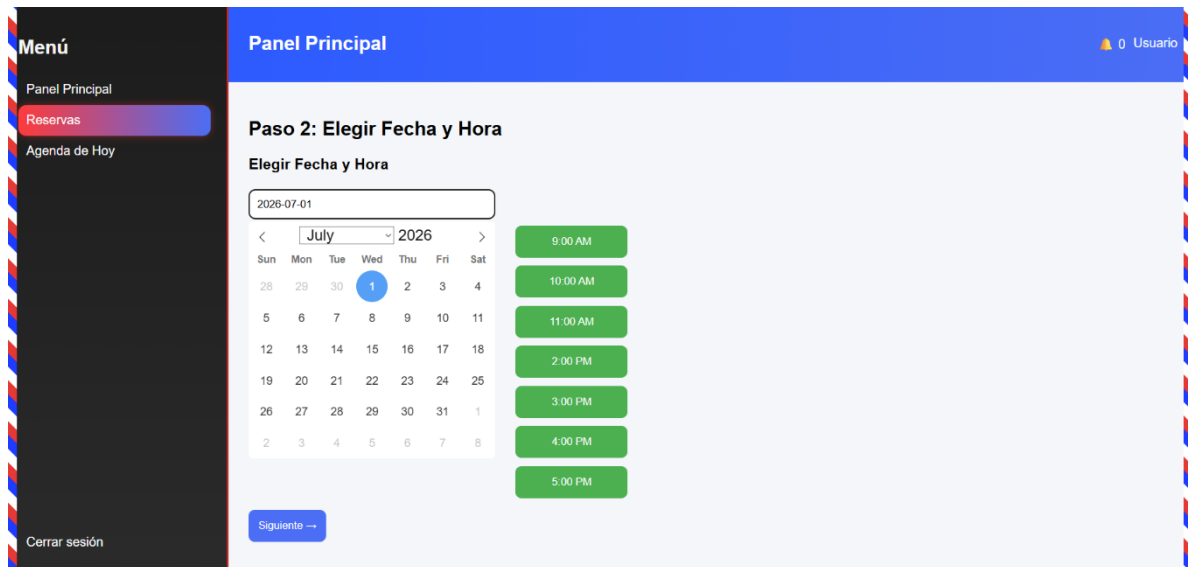
Contraseña

Correo electrónico

Registrarse

[Ir a login](#)

- Captura Reserva.



Menú

- Panel Principal
- Reservas**
- Agenda de Hoy

Panel Principal 0 Usuario

Paso 2: Elegir Fecha y Hora

Elegir Fecha y Hora

2026-07-01

< July 2026 >

Sun	Mon	Tue	Wed	Thu	Fri	Sat
28	29	30	1	2	3	4
5	6	7	8	9	10	11
12	13	14	15	16	17	18
19	20	21	22	23	24	25
26	27	28	29	30	31	1
2	3	4	5	6	7	8

9:00 AM

10:00 AM

11:00 AM

2:00 PM

3:00 PM

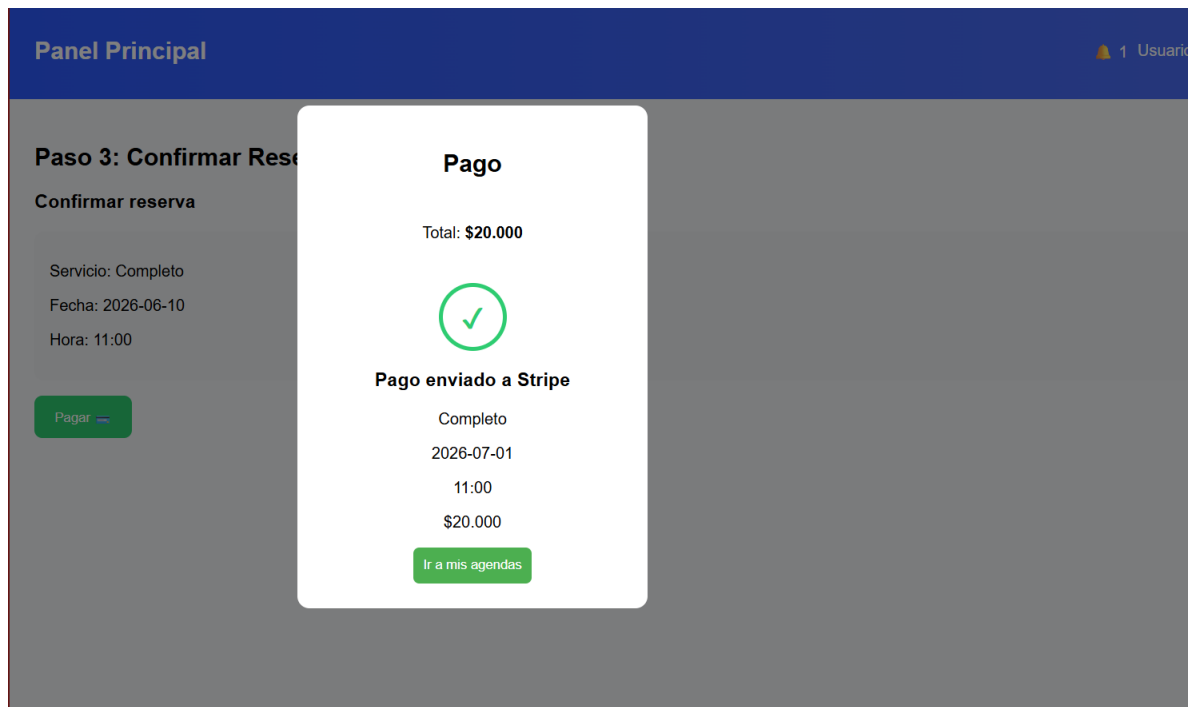
4:00 PM

5:00 PM

Siguiente →

Cerrar sesión

-
- Captura Pago.



Panel Principal 1 Usuario

Paso 3: Confirmar Reserva

Confirmar reserva

Servicio: Completo

Fecha: 2026-06-10

Hora: 11:00

Pagar

Pago

Total: \$20.000



Pago enviado a Stripe

Completo

2026-07-01

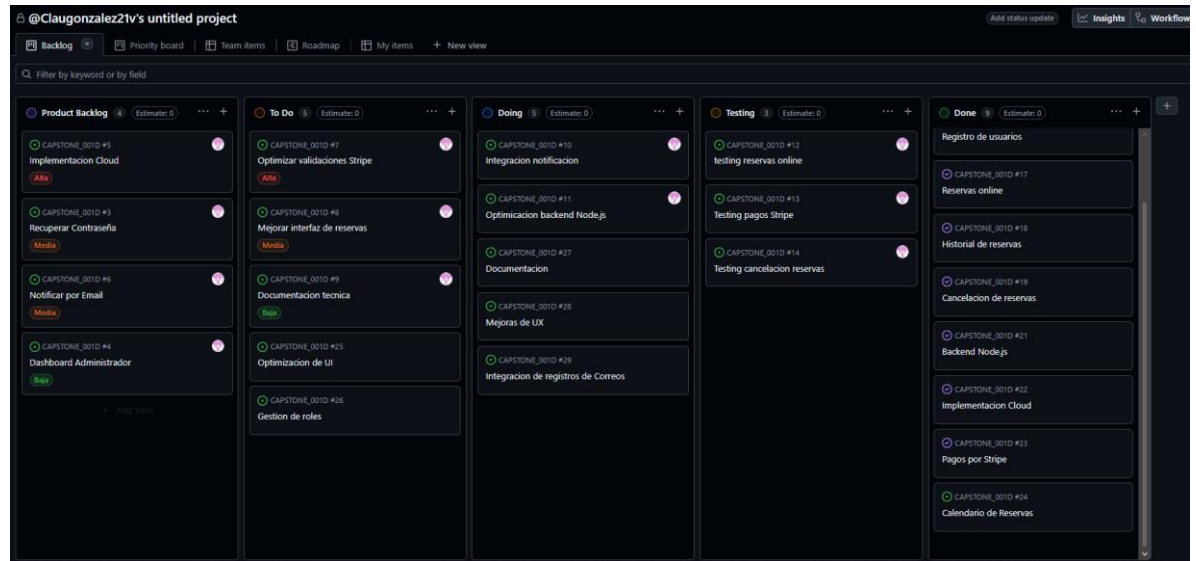
11:00

\$20.000

Ir a mis agendas

-
- Evidencia Scrum

- Tablero.



- Sprint.
- Tareas DONE.

Evidencia técnica

- GitHub.
- Commits.
- Render.

Evidencia pruebas

- Casos prueba.
- Resultados.

Evidencia despliegue

- URLs públicas.
- Logs.

Checklist final antes de entregar

Revisión	Cumple
El informe incluye portada, índice, introducción y desarrollo.	Sí
Se incorpora resumen de Fase 1.	Sí
Se explican cambios respecto a la planificación inicial.	Sí
Se incluye gestión Scrum con tablero actualizado.	Sí
Hay historias de usuario y criterios de aceptación.	Sí
Existe trazabilidad entre historia, tarea, responsable y commit.	Sí
Se incluye arquitectura técnica.	Sí
Se documenta ambiente de pruebas.	Sí
Se documenta base de datos.	Sí
Se explica respaldo/restauración de base de datos.	Sí
Se evidencia avance funcional del producto.	Sí
Se incluyen pruebas realizadas.	Sí
Se incluye estado del MVP.	Sí
Se agregan conclusiones y lecciones aprendidas.	Sí
El documento no tiene errores ortográficos evidentes.	Sí
Todos los enlaces funcionan.	Sí

Nota para los estudiantes

Este informe no debe ser tratado como un documento decorativo ni como una recopilación de capturas. Debe demostrar evidencia profesional de avance del proyecto.

Cada afirmación relevante debe estar respaldada por evidencia: tablero, GitHub, commits, capturas, pruebas, documentación técnica o producto funcional. En la defensa oral, cualquier integrante podrá ser consultado sobre cualquier tarea, historia de usuario, decisión técnica o commit asociado al avance presentado.